



پاسخ تمرین سوم

پاسخ نامه و بارم‌بندی بخش اول: سوالات تئوری و تحلیلی

پاسخ سوال ۱: تحلیل ریاضیاتی محوشدگی گرادیان و نقش Cell State در LSTM (مجموع: ۱۰۰ نمره)

الف) اثبات محوشدگی گرادیان در RNN (۵۰ نمره)

ابتدا گرادیان حالت پنهان در زمان t نسبت به زمان قبلی $(t-1)$ را محاسبه می‌کنیم. با توجه به رابطه $s_t = \tanh(Ux_t + Ws_{t-1})$ ، با استفاده از قاعده مشتق‌گیری زنجیره‌ای داریم:

$$\frac{\partial s_t}{\partial s_{t-1}} = \text{diag}(1 - s_t^2)W$$

(۱۰ نمره برای محاسبه ژاکوبین تک مرحله‌ای)

در الگوریتم پس‌انتشار در طول زمان (BPTT)، برای محاسبه اثر حالت $t-d$ بر حالت t باید ماتریس‌های ژاکوبین را متوالیاً در هم ضرب کنیم:

$$\frac{\partial s_t}{\partial s_{t-d}} = \prod_{k=t-d+1}^t \frac{\partial s_k}{\partial s_{k-1}} = \prod_{k=t-d+1}^t \text{diag}(1 - s_k^2)W$$

(۱۵ نمره برای بسط ضرب زنجیره‌ای)

حال برای بررسی اندازه این گرادیان، از نرم ماتریسی استفاده می‌کنیم:

$$\left\| \frac{\partial s_t}{\partial s_{t-d}} \right\| \leq \prod_{k=t-d+1}^t \left\| \text{diag}(1 - s_k^2) \right\| \cdot \|W\|$$

از آنجا که برد تابع $\tanh$ بین -1 و 1 است، مشتق آن $(1 - s_k^2)$ همواره مقداری کوچکتر یا مساوی ۱ دارد. بنابراین نرم ماتریس قطری همواره $1 \leq$ است. در نتیجه:

$$\left\| \frac{\partial s_t}{\partial s_{t-d}} \right\| \leq \gamma^d$$

که در آن γ بزرگترین مقدار ویژه (یا نرم) ماتریس W است. اگر $\gamma < 1$ باشد، با افزایش فاصله زمانی d ، عبارت γ^d به صورت نمایی به سمت صفر میل می‌کند و پدیده محوشدگی گرادیان رخ می‌دهد. (۲۵ نمره برای استدلال ریاضی مقادیر ویژه و رفتار نمایی تابع $\tanh$)

ب) حل مشکل توسط Cell State در LSTM (۵۰ نمره)

معادله به‌روزرسانی حافظه: $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$

برای بررسی جریان گرادیان از مسیر مستقیم (Highway) حافظه سلول، مشتق C_t نسبت به C_{t-1} را (با صرف نظر از مسیرهای غیرمستقیم گیت‌ها که از h_{t-1} می‌گذرند) محاسبه می‌کنیم:

$$\frac{\partial C_t}{\partial C_{t-1}} = \text{diag}(f_t)$$

(۲۰) نمره برای محاسبه مشتق مسیر اصلی)

در BPTT برای LSTM، جریان گرادیان در طول زمان به جای ضرب متوالی ماتریس‌های وزن (W)، به صورت زیر خواهد بود:

$$\frac{\partial C_t}{\partial C_{t-d}} = \prod_{k=t-d+1}^t \text{diag}(f_k)$$

تحلیل ریاضیاتی: به دلیل وجود عملگر جمع در معادله C_t ، تغییرات جدید ($i_t \odot \bar{C}_t$) به جای ضرب شدن، با حافظه قبلی جمع می‌شوند. همچنین گیت فراموشی (f_t) می‌تواند توسط شبکه مقداری نزدیک به ۱ به خود بگیرد. در صورتی که $f_t \approx 1$ باشد، گرادیان بدون کاهش نمایی، به صورت دست‌نخورده از طریق عملگر ضرب درایه به درایه (Pointwise Operation) به گام‌های قبلی منتقل می‌شود. این مسیر خطی، تله‌ای برای گرادیان‌ها می‌سازد که به آن چرخ‌وفلک خطای ثابت (Constant Error Carousel) می‌گویند. (۳۰) نمره برای تحلیل مفهومی عملگر جمع و نقش $f_t \approx 1$

پاسخ سوال ۲: استخراج معادلات پس‌انتشار (Backward Pass) برای گیت Reset در GRU (مجموع: ۱۰۰ نمره)

برای محاسبه $\delta W_r = \frac{\partial E}{\partial W_r}$ ، باید از انتهای شبکه (خروجی h_t) با استفاده از قاعده زنجیره‌ای به سمت W_r حرکت کنیم. فرض می‌کنیم δh_t داده شده است.

گام ۱: محاسبه گرادیان عبوری از $\tilde{h}_t$ (۲۰ نمره) با توجه به رابطه $\tilde{h}_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$:

$$\delta \tilde{h}_t = \frac{\partial E}{\partial \tilde{h}_t} = \delta h_t \odot z_t$$

گام ۲: محاسبه گرادیان آرگومان تابع $\tanh$ (۲۰ نمره) تابع فعال‌ساز $\tilde{h}_t = \tanh(a_t)$ است که در آن $a_t = W \cdot [r_t \odot h_{t-1}, x_t]$ مشتق $\tanh(x)$ برابر $1 - \tanh^2(x)$ است، پس:

$$\delta a_t = \delta \tilde{h}_t \odot (1 - \tilde{h}_t^2)$$

گام ۳: محاسبه گرادیان نسبت به خروجی گیت Reset یعنی r_t (۲۵ نمره) عبارت a_t حاصل ضرب ماتریس W در بردار متصل‌شده (Concatenated) است. اگر W را به دو بخش W_h (متناظر با بخش پنهان) و W_x (متناظر با ورودی) تجزیه کنیم، داریم $a_t = W_h \cdot (r_t \odot h_{t-1}) + W_x \cdot x_t$. ابتدا گرادیان نسبت به عبارت $(r_t \odot h_{t-1})$ را محاسبه می‌کنیم:

$$\delta(r_t \odot h_{t-1}) = W_h^T \cdot \delta a_t$$

حال برای رسیدن به r_t ، چون عملگر ضرب درایه‌ای است، h_{t-1} را فاکتور می‌گیریم:

$$\delta r_t = \frac{\partial E}{\partial r_t} = (W_h^T \cdot \delta a_t) \odot h_{t-1}$$

گام ۴: محاسبه گرادیان پیش از تابع سیگموئید در گیت Reset (۱۵ نمره) برای $r_t = \sigma(b_t)$ که $b_t = W_r \cdot [h_{t-1}, x_t]$ است، مشتق تابع سیگموئید $\sigma(x)(1 - \sigma(x))$ می‌باشد:

$$\delta b_t = \delta r_t \odot r_t \odot (1 - r_t)$$

گام ۵: محاسبه گرادین نهایی ماتریس W_r (نمره ۲۰) در بردار ورودی متصل شده $[h_{t-1}, x_t]$ ضرب شده است. در جبر خطی، مشتق نسبت به ماتریس وزن برابر با ضرب خارجی (Outer Product) گرادین خروجی لایه در ترانزاده ورودی لایه است:

$$\delta W_r = \delta b_t \otimes [h_{t-1}, x_t]^T$$

یا به فرم برداری:

$$\delta W_r = \delta b_t \cdot [h_{t-1}, x_t]^T$$

این معادله نهایی برای آپدیت وزن‌های گیت Reset است.

پاسخ‌نامه و بارم‌بندی بخش دوم: پیاده‌سازی و کدنویسی

پاسخ سوال ۳: پیاده‌سازی Coupled Input-Forget Gate LSTM از پایه با NumPy (مجموع: ۱۰۰ نمره)

الف) پیاده‌سازی کلاس LSTM و مرحله Forward Pass (۶۰ نمره)

- مقداردهی اولیه وزن‌ها (۲۰ نمره): تعریف صحیح ابعاد ماتریس‌های وزن (W_i, W_C, W_o) به ابعاد (hid-
den_size, input_size + hidden_size) و بایاس‌ها.
- توابع فعال‌ساز (۱۰ نمره): پیاده‌سازی بهینه توابع Sigmoid و Tanh.
- پیاده‌سازی معادلات کوپل شده (۳۰ نمره): محاسبه درست بردار الحاقی (Concatenation)، گیت‌ها و در نهایت فرمول $C_t = (1 - i_t) \odot C_{t-1} + i_t \odot \tilde{C}_t$ با استفاده از ضرب درایه‌ای.

ب) اعمال روی توالی زمانی مصنوعی و اعتبارسنجی ابعاد (۴۰ نمره)

- تولید داده مصنوعی (۱۰ نمره): ساخت ماتریس ورودی با ابعاد صحیح (seq_len=50, input_size).
- حلقه زمانی و استخراج حالت‌های پنهان (۲۰ نمره): پیاده‌سازی درست حلقه For روی طول توالی و ذخیره کردن h_t در هر گام.
- اعتبارسنجی خروجی (۱۰ نمره): تطابق دقیق ابعاد خروجی به شکل (seq_len, hidden_size).

کد پیاده‌سازی (قابل اجرا در گوگل کولب):

```
1 import numpy as np
2
3 # --- Activation Functions ---
4 def sigmoid(x):
5     # Clipped to avoid overflow in exp
6     x = np.clip(x, -500, 500)
7     return 1 / (1 + np.exp(-x))
8
9 class CoupledLSTM:
10     def __init__(self, input_size, hidden_size):
11         self.input_size = input_size
12         self.hidden_size = hidden_size
13
14         # Concat size: dimension of [h_{t-1}, x_t]
15         concat_size = input_size + hidden_size
16
17         # Xavier/Glorot Initialization for stable forward pass
18         scale = np.sqrt(2.0 / concat_size)
19
```

```

20     # Weights and Biases for Input Gate
21     self.W_i = np.random.randn(hidden_size, concat_size) * scale
22     self.b_i = np.zeros((hidden_size, 1))
23
24     # Weights and Biases for Cell State Candidate
25     self.W_C = np.random.randn(hidden_size, concat_size) * scale
26     self.b_C = np.zeros((hidden_size, 1))
27
28     # Weights and Biases for Output Gate
29     self.W_o = np.random.randn(hidden_size, concat_size) * scale
30     self.b_o = np.zeros((hidden_size, 1))
31
32     def forward(self, X):
33         """
34         X shape: (seq_len, input_size)
35         Returns: All hidden states with shape (seq_len, hidden_size)
36         """
37         seq_len = X.shape[0]
38
39         # Initialize Cell State and Hidden State with zeros
40         h_t = np.zeros((self.hidden_size, 1))
41         C_t = np.zeros((self.hidden_size, 1))
42
43         h_states = []
44
45         for t in range(seq_len):
46             # Extract x_t and reshape to column vector: (input_size, 1)
47             x_t = X[t].reshape(-1, 1)
48
49             # Concatenate h_{t-1} and x_t along the rows -> (hidden_size
+ input_size, 1)
50             concat_vec = np.vstack((h_t, x_t))
51
52             # 1. Input Gate
53             i_t = sigmoid(np.dot(self.W_i, concat_vec) + self.b_i)
54
55             # 2. Candidate Cell State
56             C_tilde = np.tanh(np.dot(self.W_C, concat_vec) + self.b_C)
57
58             # 3. Output Gate
59             o_t = sigmoid(np.dot(self.W_o, concat_vec) + self.b_o)
60
61             # 4. Coupled Cell State Update (f_t = 1 - i_t)
62             # Element-wise multiplication is handled by * in NumPy
63             C_t = (1 - i_t) * C_t + i_t * C_tilde
64
65             # 5. Hidden State Update
66             h_t = o_t * np.tanh(C_t)
67
68             # Store the flattened hidden state
69             h_states.append(h_t.flatten())
70

```

```

71         return np.array(h_states)
72
73     # =====
74     # Part B: Testing on Synthetic Time Series
75     # =====
76
77     # Hyperparameters
78     SEQ_LEN = 50
79     INPUT_DIM = 10
80     HIDDEN_DIM = 16
81
82     # 1. Generate Synthetic Data
83     # Shape: (50, 10)
84     X_synthetic = np.random.randn(SEQ_LEN, INPUT_DIM)
85
86     # 2. Instantiate the Custom LSTM
87     lstm_model = CoupledLSTM(input_size=INPUT_DIM, hidden_size=HIDDEN_DIM)
88
89     # 3. Perform Forward Pass
90     h_sequence = lstm_model.forward(X_synthetic)
91
92     # 4. Validation and Output
93     print(f"Input Shape: {X_synthetic.shape} -> (seq_len, input_size)")
94     print(f"Output Shape: {h_sequence.shape} -> (seq_len, hidden_size)")
95     print("\nFirst 5 values of the final hidden state (h_50):")
96     print(h_sequence[-1][:5])

```

Code 1: Implementation of Coupled Input-Forget Gate LSTM

پاسخ‌نامه و بارم‌بندی بخش دوم: پیاده‌سازی و کدنویسی (ادامه)

پاسخ سوال ۴: پیاده‌سازی برداری پردازش دسته‌ای و پس‌انتشار در زمان (BPTT) (مجموع: ۱۰۰ نمره)
 الف) ساختار کلاس RNN و ترفند Concatenation در Forward Pass (۲۵ نمره)

- پشتیبانی از **Batching** و مقداردهی (۱۰ نمره): درک صحیح از ابعاد سه‌بعدی ورودی، $(batch, seq_len, features)$ و تخصیص ابعاد درست برای ماتریس وزن واحد $W_{combined}$.
- پیاده‌سازی ترفند **Concatenation** (۱۵ نمره): استفاده از دستور `np.concatenate` یا `np.hstack` در حلقه زمانی برای ادغام h_{t-1} و x_t پیش از ضرب ماتریسی.

ب) پیاده‌سازی BPTT و Gradient Clipping (۳۵ نمره)

- انتقال گرادیان در طول زمان (۲۰ نمره): حرکت صحیح رو به عقب در حلقه (reversed loop)، محاسبه مشتق تابع $\tanh(h_t^2)$ و انباشت (Accumulation) گرادیان‌های ماتریس وزن در متغیرهای db و dW .
- برش گرادیان (۱۵ نمره): پیاده‌سازی دستی مکانیزم Clipping روی گرادیان‌ها پیش از اعمال گرادیان کاهش (جلوگیری از خطای NaN).

ج و د) دیتاست مصنوعی، حلقه آموزش و رسم نمودار (۴۰ نمره)

- ساخت دیتاست (۱۰ نمره): تولید داده‌های باینری و استخراج برچسب بر اساس زوج یا فرد بودن مجموع (مسئله Parity).

- حلقه آموزش و Loss (۲۰ نمره): نوشتن یک حلقه تمیز با قابلیت محاسبه Binary Cross-Entropy Loss.
- بهینه‌سازی و رسم (۱۰ نمره): همگرایی مناسب در زیر چند ثانیه روی CPU و رسم نمودار خطای نزولی با Matplotlib.

کد پیاده‌سازی (قابل اجرا در گوگل کولب):

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import time
4
5 # --- Helper Functions ---
6 def sigmoid(x):
7     # Clipped to prevent overflow
8     return 1 / (1 + np.exp(-np.clip(x, -250, 250)))
9
10 def bce_loss(y_pred, y_true):
11     # Add epsilon to prevent log(0)
12     eps = 1e-15
13     y_pred = np.clip(y_pred, eps, 1 - eps)
14     return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 -
15     y_pred))
16
17 # --- Batched RNN Class ---
18 class BatchedVanillaRNN:
19     def __init__(self, input_size, hidden_size, output_size=1):
20         self.hidden_size = hidden_size
21         self.input_size = input_size
22
23         # Xavier initialization for stability
24         concat_size = hidden_size + input_size
25         scale_rnn = np.sqrt(2.0 / concat_size)
26         scale_out = np.sqrt(2.0 / hidden_size)
27
28         # W_combined handles both previous hidden state and current input
29         self.W = np.random.randn(concat_size, hidden_size) * scale_rnn
30         self.b = np.zeros((1, hidden_size))
31
32         # Output layer weights (Many-to-One)
33         self.V = np.random.randn(hidden_size, output_size) * scale_out
34         self.c = np.zeros((1, output_size))
35
36     def forward(self, X):
37         """
38         X shape: (batch_size, seq_len, input_size)
39         """
40         self.batch_size, self.seq_len, _ = X.shape
41
42         # Store hidden states and inputs for BPTT
43         # H shape: (seq_len + 1, batch_size, hidden_size)
44         self.H = np.zeros((self.seq_len + 1, self.batch_size, self.
45         hidden_size))

```

```

44     self.concat_inputs = np.zeros((self.seq_len, self.batch_size,
45                                     self.hidden_size + self.input_size
46                                     ))
47
48     for t in range(self.seq_len):
49         x_t = X[:, t, :]
50         h_prev = self.H[t]
51
52         # The Concatenation Trick [h_{t-1}, x_t]
53         concat_in = np.concatenate((h_prev, x_t), axis=1)
54         self.concat_inputs[t] = concat_in
55
56         # Forward step
57         self.H[t+1] = np.tanh(np.dot(concat_in, self.W) + self.b)
58
59         # Many-to-One: Use only the last hidden state for prediction
60         h_last = self.H[-1]
61         self.y_pred = sigmoid(np.dot(h_last, self.V) + self.c)
62     return self.y_pred
63
64 def backward(self, y_true, lr=0.01, clip_val=5.0):
65     # Derivative of BCE Loss + Sigmoid simplifies to: (y_pred -
66     y_true)
67     dy = (self.y_pred - y_true) / self.batch_size
68
69     # Output layer gradients
70     dV = np.dot(self.H[-1].T, dy)
71     dc = np.sum(dy, axis=0, keepdims=True)
72
73     # Gradient entering the recurrent part from the output layer
74     dh_next = np.dot(dy, self.V.T)
75
76     dW = np.zeros_like(self.W)
77     db = np.zeros_like(self.b)
78
79     # BPTT Loop (Moving backwards in time)
80     for t in reversed(range(self.seq_len)):
81         h_t = self.H[t+1]
82
83         # Derivative of tanh: (1 - tanh^2)
84         dtanh = 1 - h_t ** 2
85         dz = dh_next * dtanh
86
87         concat_in = self.concat_inputs[t]
88
89         # Accumulate weight gradients
90         dW += np.dot(concat_in.T, dz)
91         db += np.sum(dz, axis=0, keepdims=True)
92
93         # Gradient with respect to concatenated input [h_{t-1}, x_t]
94         dconcat = np.dot(dz, self.W.T)

```

```

94         # Extract gradient for the previous hidden state (first '
hidden_size' columns)
95         dh_next = dconcat[:, :self.hidden_size]
96
97         # Explicit Manual Gradient Clipping
98         for grad in [dW, db, dV, dc]:
99             np.clip(grad, -clip_val, clip_val, out=grad)
100
101         # SGD Parameter Update
102         self.W -= lr * dW
103         self.b -= lr * db
104         self.V -= lr * dV
105         self.c -= lr * dc
106
107 # --- Main Training Routine ---
108 if __name__ == "__main__":
109     # 1. Create Synthetic Parity Dataset (Many-to-One)
110     # Goal: Determine if the number of 1s in a sequence of length 5 is
odd or even
111     np.random.seed(42)
112     NUM_SAMPLES = 2000
113     SEQ_LEN = 5
114
115     # Shape: (2000, 5, 1)
116     X_data = np.random.randint(0, 2, size=(NUM_SAMPLES, SEQ_LEN, 1)).
astype(float)
117     # Shape: (2000, 1) - Label is 1 if sum is odd, 0 if even
118     y_data = (np.sum(X_data, axis=1) % 2).astype(float)
119
120     # 2. Initialize Model
121     model = BatchedVanillaRNN(input_size=1, hidden_size=16, output_size
=1)
122
123     # 3. Training Loop
124     EPOCHS = 500
125     learning_rate = 0.5
126     loss_history = []
127
128     print("Starting training...")
129     start_time = time.time()
130
131     for epoch in range(EPOCHS):
132         # Forward pass on the entire batch
133         y_pred = model.forward(X_data)
134
135         # Calculate Loss
136         loss = bce_loss(y_pred, y_data)
137         loss_history.append(loss)
138
139         # Backward pass and update
140         model.backward(y_data, lr=learning_rate, clip_val=3.0)
141

```



```

142         if (epoch + 1) % 100 == 0:
143             predictions = (y_pred >= 0.5).astype(float)
144             accuracy = np.mean(predictions == y_data)
145             print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {loss:.4f} |
Accuracy: {accuracy*100:.1f}%")
146
147         end_time = time.time()
148         print(f"Training completed in {end_time - start_time:.2f} seconds.")
149
150         # 4. Plotting the Learning Curve
151         plt.figure(figsize=(8, 5))
152         plt.plot(loss_history, color='blue', linewidth=2)
153         plt.title('BCE Loss Reduction over Epochs (Vanilla RNN BPTT)',
154                 fontsize=14)
155         plt.xlabel('Epochs', fontsize=12)
156         plt.ylabel('Loss', fontsize=12)
157         plt.grid(True)
158         plt.show()

```

Code 2: Batched Vanilla RNN with BPTT and Gradient Clipping

پاسخ‌نامه و بارم‌بندی بخش دوم: پیاده‌سازی و کدنویسی (ادامه)

پاسخ سوال ۵: پیاده‌سازی حرفه‌ای و ترفندهای آموزش RNN‌ها در PyTorch (مجموع: ۱۰۰ نمره)

الف) معماری مدل و مدیریت طول متغیر توالی‌ها (۴۰ نمره)

- تعریف صحیح معماری (۱۰ نمره): ارث‌بری درست از nn.Module، پیاده‌سازی LSTM با پارامترهای bidirectional=True، تنظیم num_layers و dropout، و تعریف لایه Linear با ابعاد ورودی ۲ برابر (به دلیل دوطرفه بودن).
- استفاده از pack_padded_sequence (۱۵ نمره): اعمال درست این تابع برای فشردن توالی‌ها پیش از ورود به LSTM جهت جلوگیری از هدررفت محاسبات روی پدینگ‌ها.
- استفاده از pad_packed_sequence و استخراج ویژگی (۱۵ نمره): بازگرداندن خروجی به فرمت تنسور پدشده (طبق صورت سوال) و استخراج صحیح آخرین حالت پنهان مسیرهای رفت و برگشت از h_n و Concatenation آن‌ها برای لایه کلاسیفایر.

ب) حلقه آموزش پایدار و ترفندها (۶۰ نمره)

- مقداردهی اولیه متعامد (Orthogonal Init) (۱۵ نمره): نوشتن تابعی که روی پارامترهای مدل پیمایش کرده و ماتریس‌های وزن تکرارشونده (weight_hh) را با nn.init.orthogonal_ مقداردهی کند.
- برش گرادینان (Gradient Clipping) (۱۵ نمره): استفاده صحیح از torch.nn.utils.clip_grad_norm_ بعد از loss.backward() و قبل از optimizer.step().
- مدیریت State و Detach کردن (۱۵ نمره): مقداردهی اولیه حالت پنهان در هر بیچ و استفاده از متد detach(). برای جلوگیری از رشد گراف محاسباتی و خطای Out of Memory.
- تولید داده مصنوعی و اجرای حلقه (۱۵ نمره): تولید دسته‌ای از توالی‌ها با طول متغیر، پدینگ آن‌ها با pad_sequence، محاسبه Loss و اجرای کامل حلقه روی CPU.

کد پیاده‌سازی (قابل اجرا در گوگل کولب):

```

1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence,
  pad_sequence
5
6 # =====
7 # Part A: Model Architecture
8 # =====
9 class RobustBiLSTM(nn.Module):
10     def __init__(self, input_size, hidden_size, num_layers, num_classes,
11                 dropout_prob=0.5):
12         super(RobustBiLSTM, self).__init__()
13         self.hidden_size = hidden_size
14         self.num_layers = num_layers
15
16         # Stacked Bidirectional LSTM
17         self.lstm = nn.LSTM(input_size=input_size,
18                             hidden_size=hidden_size,
19                             num_layers=num_layers,
20                             batch_first=True,
21                             bidirectional=True,
22                             dropout=dropout_prob if num_layers > 1 else
23
24         0)
25
26         # Classifier Layer: hidden_size * 2 because of bidirectionality
27         self.fc = nn.Linear(hidden_size * 2, num_classes)
28
29     def forward(self, x, lengths, hidden):
30         # 1. Pack the padded sequence to skip padding computations
31         # enforce_sorted=False allows passing un-sorted batches (requires
32         # PyTorch 1.1+)
33         packed_x = pack_padded_sequence(x, lengths, batch_first=True,
34                                         enforce_sorted=False)
35
36         # 2. Pass through LSTM
37         packed_out, hidden = self.lstm(packed_x, hidden)
38
39         # 3. Unpack the sequence (Required by the prompt)
40         # out shape: (batch_size, max_seq_len, hidden_size * 2)
41         out, out_lengths = pad_packed_sequence(packed_out, batch_first=
42         True)
43
44         # 4. Extract final hidden states for Sequence Classification
45         h_n, c_n = hidden
46         # h_n shape: (num_layers * 2, batch_size, hidden_size)
47         # We need the last layer's forward and backward states
48         h_forward = h_n[-2, :, :] # Last layer, forward direction
49         h_backward = h_n[-1, :, :] # Last layer, backward direction
50
51         # Concatenate forward and backward states
52         final_h = torch.cat((h_forward, h_backward), dim=1)

```

```

47
48     # 5. Pass to fully connected layer
49     logits = self.fc(final_h)
50
51     return logits, hidden
52
53 # =====
54 # Part B: Training Loop & Tricks
55 # =====
56
57 # Trick 2: Orthogonal Initialization
58 def init_weights_orthogonal(m):
59     """
60     Applies Orthogonal initialization to recurrent weights to prevent
61     vanishing/exploding gradients in BPTT.
62     """
63     if type(m) == nn.LSTM:
64         for name, param in m.named_parameters():
65             if 'weight_hh' in name: # Recurrent weights
66                 nn.init.orthogonal_(param)
67             elif 'weight_ih' in name: # Input weights
68                 nn.init.xavier_uniform_(param)
69             elif 'bias' in name:
70                 nn.init.zeros_(param)
71                 # Optional: Initialize forget gate bias to 1 for better
72                 # early learning
73                 n = param.size(0)
74                 param.data[n//4:n//2].fill_(1.0)
75
76 # Trick 3: Hidden State Management
77 def detach_hidden(hidden):
78     """
79     Detaches hidden states from their history to prevent the computation
80     graph from growing indefinitely across batches.
81     """
82     if isinstance(hidden, tuple):
83         return tuple(detach_hidden(v) for v in hidden)
84     else:
85         return hidden.detach()
86
87 if __name__ == "__main__":
88     # Hyperparameters
89     INPUT_SIZE = 10
90     HIDDEN_SIZE = 32
91     NUM_LAYERS = 2
92     NUM_CLASSES = 2
93     BATCH_SIZE = 4
94     EPOCHS = 10
95
96     # Initialize Model & Apply Orthogonal Weights
97     model = RobustBiLSTM(INPUT_SIZE, HIDDEN_SIZE, NUM_LAYERS, NUM_CLASSES
98 )

```

```

97     model.apply(init_weights_orthogonal)
98
99     criterion = nn.CrossEntropyLoss()
100    optimizer = optim.Adam(model.parameters(), lr=0.001)
101
102    # Create Synthetic Variable-Length Dataset
103    # 4 sequences of lengths 5, 8, 3, and 6
104    seq_lengths = torch.tensor([5, 8, 3, 6])
105    sequences = [torch.randn(1, INPUT_SIZE) for l in seq_lengths]
106    labels = torch.tensor([1, 0, 1, 0]) # Random binary targets
107
108    # Pad sequences to create a valid tensor batch: (4, 8, 10)
109    padded_batch = pad_sequence(sequences, batch_first=True)
110
111    print("Starting Training...")
112    model.train()
113
114    # Initial hidden state (None lets PyTorch initialize with zeros)
115    # But to demonstrate Trick 3, we initialize it explicitly
116    num_directions = 2
117    h0 = torch.zeros(NUM_LAYERS * num_directions, BATCH_SIZE, HIDDEN_SIZE
118    )
119    c0 = torch.zeros(NUM_LAYERS * num_directions, BATCH_SIZE, HIDDEN_SIZE
120    )
121    hidden = (h0, c0)
122
123    for epoch in range(EPOCHS):
124        optimizer.zero_grad()
125
126        # Trick 3: Detach hidden states before each batch
127        hidden = detach_hidden(hidden)
128
129        # Forward Pass
130        logits, hidden = model(padded_batch, seq_lengths, hidden)
131
132        # Compute Loss
133        loss = criterion(logits, labels)
134
135        # Backward Pass
136        loss.backward()
137
138        # Trick 1: Gradient Clipping
139        # Prevents exploding gradients by capping the norm to a max value
140        # (e.g., 5.0)
141        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
142
143        # Update weights
144        optimizer.step()
145
146        print(f"Epoch [{epoch+1}/{EPOCHS}], Loss: {loss.item():.4f}")

```

145

```
print("Training Complete!")
```

Code 3: Robust BiLSTM with Variable Length Handling and Stable Training Tricks